Java Programming Final Project Instructions

Overview

Students will develop ONE of the following systems using Java:

1. Restaurant Ordering System (Fast Food/Coffee Shop)
2. Ticketing System (Bus/Air/Train)
3. Inventory Management System (School Supplies/Computer Laboratory)

The project must demonstrate understanding of core programming concepts, including all the lessons covered by the subject CC 104, which are:

1. Methods
2. Arrays
3. ArrayLists
4. exception handling
5. file handling, and
6. Basic GUI design using Swing.

General Requirements (All Projects)

Required Technical Components. Your system MUST include:

1. At least 3 user-defined methods
2. At least 2 classes
3. At least 1 array
4. At least 1 ArrayList
5. Try–catch exception handling
6. File input and output (save + load data)
7. Graphical User Interface using Swing

File Handling Requirement

Your program must: - Save data to a file when user exits or presses Save - Load existing data when program starts

---

Error Handling Requirement

You must handle possible errors such as: - Invalid numeric input - Missing files - Empty fields - Invalid selections

---

Project Topic:

The **Restaurant Ordering System** is a Java-based application designed to manage customer food orders efficiently. The system should be able to display a menu, allow users to add and remove orders, compute the total bill, show a receipt, and save the receipt to a file for record-keeping. To implement these features, students are encouraged to create methods such as addItem(), removeItem(), calculateTotal(), printReceipt(), and saveOrder() to organize the program's functionality. In terms of data structures, an array can be used to store the list of menu items, while an ArrayList can manage the customer's selected orders dynamically. For additional enhancement, students may include bonus features such as a discount system, support for multiple tables, and automatic tax computation to make the application more realistic and advanced.

The **Ticketing System** is a Java-based application designed to manage passenger reservations for air, bus, or train transportation (choose one only). The system should allow users to view available trips or schedules, book tickets, cancel bookings, display a list of passengers, and save booking data for future use. To organize program functionality, students are encouraged to implement methods such as searchTrip() (or searchFlight()), bookSeat(), cancelBooking(), and displayPassengers(), which handle the core operations of the system. For data storage, an array may be used to store the list of available schedules or routes, while an ArrayList can manage passenger records dynamically as bookings are added or removed. To enhance the system, students may incorporate bonus features such as seat selection, boarding pass or ticket display, and class-based pricing (for example, Economy and Business), making the application more realistic and closer to real-world ticketing systems.

The **Inventory Management System** is a Java-based application designed to manage and monitor items in environments such as a computer laboratory or school supply inventory management system. The system should allow users to add products or equipment, update stock quantities, delete items, search for specific products, generate low-stock alerts, and save inventory data for future access. To organize program functionality, students are encouraged to implement methods such as addProduct(), updateStock(), deleteProduct(), searchItem(), and checkLowStock(), each responsible for handling key system operations. For data structures, an ArrayList can be used to store and manage the list of products dynamically, while an array may be used for storing stock history, category lists, or predefined item types. To make the project more advanced and realistic, students may include bonus features such as generating usage or sales reports, tracking suppliers or item sources, and maintaining a history log of inventory transactions.

Submission Requirements:

Students must submit the following: the source code folder, compiled program, sample data file, and complete documentation.

1. The project demonstrations will be held on **April 6–8, 2026**.
2. The written final examination will take place on **April 9–10, 2026**.
3. Grade consultations are scheduled for **April 13–14, 2026**, and
4. The removal examination will be conducted on **April 15, 2026**.

The documentation must include a clear program description, a flowchart or logic diagram illustrating the system's process, screenshots of the interface, an explanation of the methods used in the program, and a reflection discussing what the student learned from developing the project. The document should be formatted on A4 paper using Times New Roman, size 12, single-spaced, and must be at least 5 pages but no more than 10 pages.

Criteria and Rating	**Excellent (20 – 17)**	**Very Satisfactory (16 – 13)**	**Satisfactory (12 – 9)**	**Needs Improvement (8 – 5)**	**Poor (4 – 0)**	**Score**	
Individual presentation	The student shows presentation skills and confidence in the delivery of topics and answering of questions without looking on their notes	The student shows presentation skills and confidence in the delivery of topics and answering of questions, but looks at their notes	The student shows presentation skills, but lacks confidence in the delivery of topics and answering questions, as but look at their notes	The students don't show their presentation skills	No presentation at all/ did not talk or anything	**S1**	5
						S2	10
						S3	30
						S4	30
Functionality	The completed program runs without errors. It meets all the specifications and works for all test data.	The completed program is tested and runs, but does not meet all the specifications and/or work for all test data.	The completed program runs but has logical errors.	The program is incomplete and does not run.	The program is not working		
Program Design/flow	The program uses appropriate control	The program design generally uses	Not all selected structures are appropriate.	A few of the selected structures are appropriate.	The program design is completely		

	structures. The data structure used is appropriate for organizing, processing, retrieving, and storing data.	appropriate structures. Program elements exhibit good design.	Some of the program elements are appropriately designed.	Program elements are not well designed.	horrible	
User interface	User interaction meets the specifications. Good prompts, headings, data validation, and labels formatting.	User interaction generally meets the specifications and is acceptable to the user.	User interaction meets the minimum specifications but does not improve usability. Prompts are confusing, and output is difficult to read, with no data validation.	User interaction is incomplete and does not specify. No user interaction is possible.	The user interface is completely unattractive	
					Total Project Score	/ 80

Sample Images:

Ticketing System

Inventory System

Ordering System